

DYNAMIC PROGRAMMING & GREEDY ALGORITHMS

Greedy Approach

Definition:

make the most advantageous local choice at each step to find a global optimum

Use Cases:

Network design, such as building minimal spanning trees with Prim's or Kruskal's algorithms.

Scheduling problems, like selecting jobs that finish the earliest or have the shortest duration.

Resource allocation, ensuring optimal usage of limited resources under constraints.

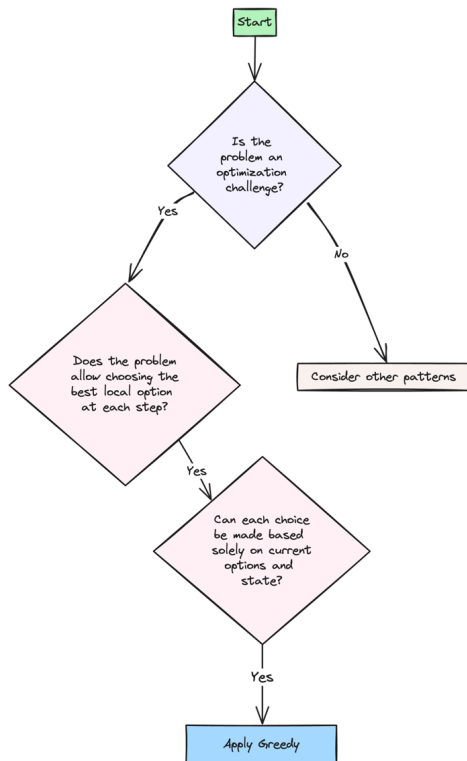
Advantages:

Efficiency: Often provides a simple and fast solution for complex problems.

Ease of Implementation: Less complex than dynamic programming or backtracking solutions.

Effective for Many Problems: Offers optimal solutions for problems like spanning trees, scheduling, and others under certain conditions.

Does My Problem Match This Pattern?



Dynamic Programming

Definition:

solving complex problems by breaking them down into simpler subproblems and solving each subproblem just once, storing its solutions

Use Cases:

Optimization problems where decisions are made to achieve the best outcome (e.g., minimizing costs, maximizing profits).

Problems that can be broken down into overlapping subproblems, where the solution relies on solutions to smaller instances.

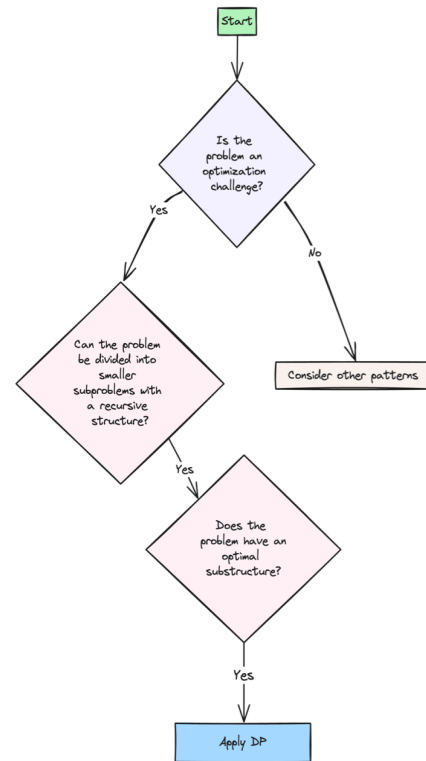
Advantages:

Efficiency: Solves each smaller subproblem only once and stores its answer in a table, avoiding the work of recalculating it, thus saving computation time.

Optimality: Ensures that the solution to the problem is optimal by building on optimal solutions to its subproblems.

Applicability: Useful in a wide range of applications from genetics to computer graphics and operations research.

Does My Problem Match This Pattern?



PROBLEM - 1

198. House Robber

Solved

Medium Topics Companies

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.

Example 2:

Input: `nums = [2,7,9,3,1]`

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).
Total amount you can rob = 2 + 9 + 1 = 12.

Constraints:

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 400`

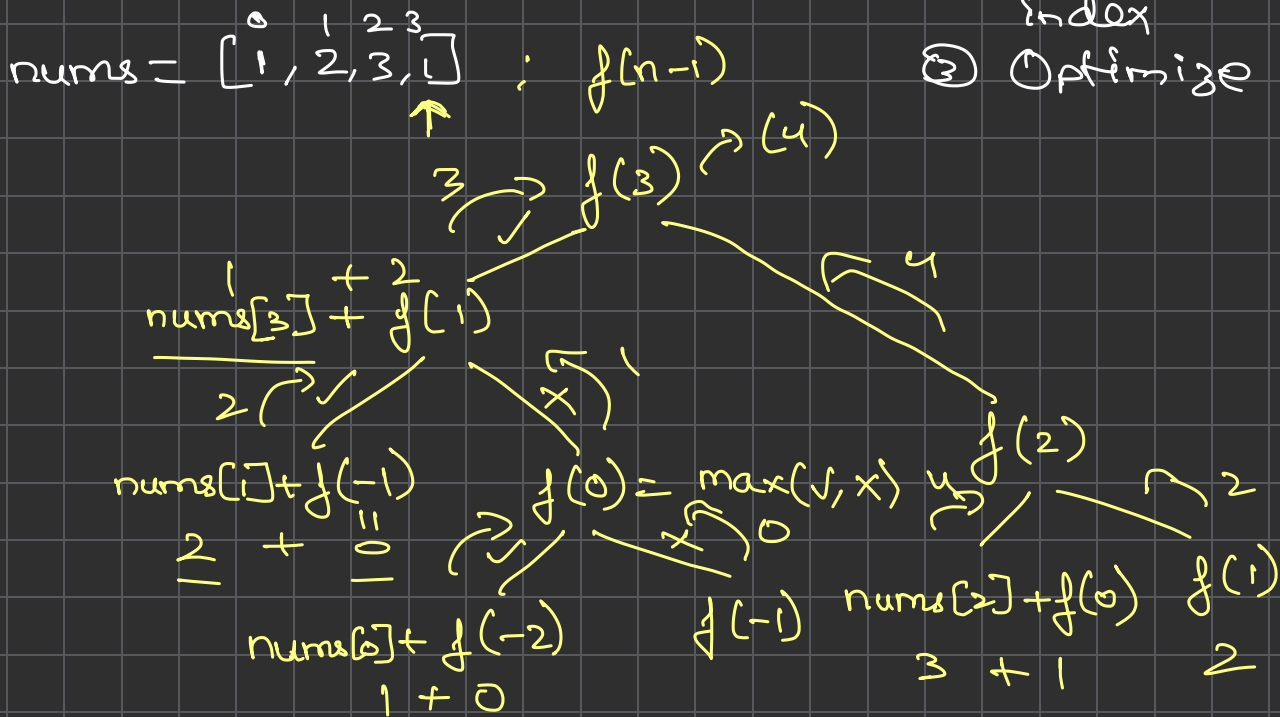
`nums = [1, 2, 3, 1]`
`output = 4`

`[1, 2, 3, 1]`
houses: 1 2 3 4
case: ✓ x ✓ x } 4
case: x ✓ x ✓ } 3

① express the problem in terms of indices

② do all stuff on that index

③ Optimize



pseudocode:

Recursive { $f(index)$

if (`index < 0`) return 0;

pick case

`pick = nums[index] + f(index - 2)`

not pick case

$f(n-1)$ } func call to get the ans

$$\text{not_pick} = 0 + f(\text{index} - 1)$$

$$\text{return max(pick, not_pick)}$$

}

$$T.C = \underline{2^n}$$

D.P. recursive:

Top-down

$$\underline{dp} = \underline{[-1]} * \text{len(nums)}$$

{ f(index)

if (index < 0) return 0;

if (index == 0) return nums[index];

if (dp[index] != -1) return dp[index];

pick case

$$\text{pick} = \text{nums}[\text{index}] + f(n-2)$$

not pick

$$\text{not_pick} = 0 + f(n-1)$$

$$\text{dp}[\text{index}] = \text{max}(\text{pick}, \text{not_pick})$$

$$\text{return dp}[\text{index}]$$

}

$$T.C = O(n)$$

$$S.C = O(n) + \underline{O(n)}$$

stack space

PROBLEM-2

253. Meeting Rooms II Premium

Solved

Medium Topics Companies Hint

Given an array of meeting time intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of conference rooms required.

Example 1:

Input: `intervals = [[0,30],[5,10],[15,20]]`
Output: 2

Example 2:

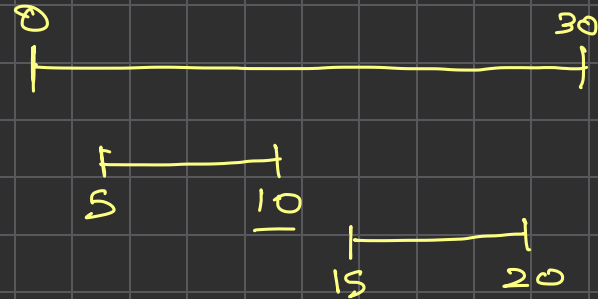
Input: `intervals = [[7,10],[2,4]]`
Output: 1

Constraints:

- $1 \leq \text{intervals.length} \leq 10^4$
- $0 \leq \text{start}_i < \text{end}_i \leq 10^6$

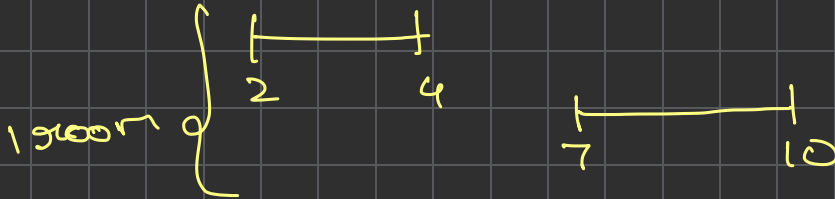
intervals =

[0,30], [5,10], [15,20]



min_rooms = 2

intervals = [[7,10], [2,4]]



intervals = $\left[\begin{smallmatrix} s & e \\ [0, 30] \end{smallmatrix}, \begin{smallmatrix} s & e \\ [5, 10] \end{smallmatrix}, \begin{smallmatrix} s & e \\ [15, 20] \end{smallmatrix} \right]$

st = [0, 5, 15]

et = [30, 10, 20]

* * * ↓

→ sorted-st = [0, 5, 15]

sorted-et = [~~15~~, 20, 30]
 0 1 2
 * ↑

min_rooms = 0
 (max)

count = ~~1~~ * ~~2~~ * ~~3~~ * 2

max = max(max, count)

TC = $O(n)$

SC = $O(n) + O(n)$

PROBLEM - 3

1143. Longest Common Subsequence

Solved

Medium Topics Companies Hint

Given two strings `text1` and `text2`, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A **subsequence** of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

For example, "ace" is a subsequence of "abcde".

A **common subsequence** of two strings is a subsequence that is common to both strings.

Example 1:

Input: `text1 = "abcde"`, `text2 = "ace"`
Output: 3

Explanation: The longest common subsequence is "ace" and its length is 3.

Example 2:

Input: `text1 = "abc"`, `text2 = "abc"`
Output: 3

Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:

Input: `text1 = "abc"`, `text2 = "def"`
Output: 0

Explanation: There is no such common subsequence, so the result is 0.

Constraints:

- $1 \leq \text{text1.length}, \text{text2.length} \leq 1000$
- `text1` and `text2` consist of only lowercase English characters.

Ex:

`text1 = "abcde"`
`text2 = "ace"` } ans = 3

Recursion:

- Express in terms of indices
- Do all stuff on that indices
- Optimize

`text1 = "abcde"`; `text2 = "ace"`
 n_1 n_2

$f(n_1 - 1, n_2 - 1)$

$f(i_1, i_2)$

(4, 2) (abcde | ace)
↓

1 + f(3, 1) (abcd | ac)

match case

$\text{if } (t_1[i_1] == t_2[i_2])$

return

$1 + f(i_1 - 1, i_2 - 2)$

not match

return

$\max(f(i_1 - 1, i_2), f(i_1, i_2 - 1))$

Recursion: $T = O(2^{n_1 + n_2})$
 $SC = O(n_1 + n_2)$

(abc | ac) f(2, 1)

(3, 0) (abcd | a)

1 + f(1, 0) (ab | a)

(abc | a) f(3, -1) f(2, 0)

f(0, 0) (a | a)

f(1, -1)

f(1, 0)

f(2, -1)

1 + f(-1, -1)

$$DP: TC = O(n_1 \times n_2)$$

$$SC = O(n_1 \times n_2) + O(n_1 + n_2)$$

Top-down (D.P using recursion)

$$dp = [[-1] * n_1] * n_2$$

$$f(i_1, i_2)$$

{ if ($i_1 < 0$ or $i_2 < 0$) return 0;

if ($dp[i_1][i_2] \neq -1$) return $dp[i_1][i_2]$;

match case

if ($t_1[i_1] == t_2[i_2]$):

$$dp[i_1][i_2] = 1 + f(i_1 - 1, i_2 - 1)$$

else:

not match

$$dp[i_1][i_2] = \max(f(i_1 - 1, i_2), f(i_1, i_2 - 1))$$

return $dp[i_1, i_2]$;

}

PROBLEM - 4

435. Non-overlapping Intervals

Solved 

Medium  Topics  Companies

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the *minimum number of intervals you need to remove to make the rest of the intervals non-overlapping*.

Note that intervals which only touch at a point are **non-overlapping**. For example, `[1, 2]` and `[2, 3]` are non-overlapping.

Example 1:

Input: `intervals = [[1,2],[2,3],[3,4],[1,3]]`

Output: 1

Explanation: `[1,3]` can be removed and the rest of the intervals are non-overlapping.

Example 2:

Input: `intervals = [[1,2],[1,2],[1,2]]`

Output: 2

Explanation: You need to remove two `[1,2]` to make the rest of the intervals non-overlapping.

Example 3:

Input: `intervals = [[1,2],[2,3]]`

Output: 0

Explanation: You don't need to remove any of the intervals since they're already non-overlapping.

Constraints:

- `1 <= intervals.length <= 105`
- `intervals[i].length == 2`
- `-5 * 104 <= starti < endi <= 5 * 104`

